Project Proposal: Home Energy Coach

A Personalized Green Habit Recommender for Students and Renters in Shared Flats

Muhammad Saad Zafar[1541717], Asif Newaz[1446896], Mazba Uddin[1445693],
Madhusudan Ambulkar[1572416], Nazmul Alam[1471706], Md. Rabiul Hasan Tanvir[1444348]
Frankfurt University of Applied Sciences
High Integrity Systems, Winter Semester 2026/27
Supervisor: Prof. Dr. Christina Andersson

***Abstract*—This proposal describes the plan for *Home Energy Coach*, the team's Personalized Green Habit Recommender for the High Integrity Systems Project at Frankfurt University of Applied Sciences (WS 2026/27). The app is aimed at students and renters in shared flats (WGs). They cannot change their building's insulation or heating system, but they can change their daily habits. Users fill in a short energy profile, get a ranked list of habits that suit their flat and schedule, commit to one small goal at a time and are reminded when it is time to act. The main feature is the *Green Hours planner*. It uses live generation data for the German electricity market from the Bundesnetzagentur's SMARD platform to tell each user the lowest-CO_2 hours for laundry, the dishwasher and charging, limited to the hours they are actually at home. The native iOS app (SwiftUI) talks to a FastAPI back-end, which handles the application logic and passes all data analysis to an R analytics service exposed through the `plumber` package. That analysis covers the hourly CO_2 forecast, recommendation ranking, association-rule mining, user clustering and the evaluation of nudges. User data is stored in PostgreSQL, and the server side runs in Docker containers. Following the course's green software requirement, the team measures the app's own energy and resource use from the first prototype onwards, reduces it in every sprint and shows the results in the app. The project runs as five Scrum sprints with rotating Scrum Master and Product Owner roles, and all progress is tracked in Jira.**

I. Introduction

Private households account for a substantial share of Germany's final energy consumption. Within a household, more than two thirds of that energy goes into space heating, well ahead of hot water, cooking and electrical appliances [6]. Most energy advice is therefore about the building: insulate, replace the boiler, install a heat pump. Students and other renters, and residents of shared flats (Wohngemeinschaften, WGs) in particular, cannot act on any of this. They do not own the building, rarely choose the heating system or the electricity contract, and often never see their own consumption because the flat gets a single bill. What they *can* change is their daily behaviour: how they heat and air their rooms, how long they shower, which devices they leave on standby and, increasingly important, *when* their appliances run.

Timing matters because the CO_2 intensity of German electricity changes a lot within a day as the share of wind and solar power rises and falls from hour to hour. The Bundesnetzagentur publishes actual and forecast generation per energy source, together with actual consumption for Germany, on its SMARD platform, free of charge and in machine-readable form [3]. From this data the hourly CO_2 intensity can be calculated, and a washing-machine load, a dishwasher run or a laptop charge can be moved into the cleanest hours, as long as the user is at home then. This is what the *Green Hours planner* does.

Information alone rarely changes behaviour, and the gap between good intentions and action is well documented. Home Energy Coach is therefore built to form habits rather than to inform. It follows the Fogg Behavior Model, in which a behaviour happens only when motivation, ability and a prompt come together at the same moment [9], and the habit loop of cue, routine and reward. Users commit to one small, concrete goal at a time, an implementation intention of the form "when X, I do Y" [10]. The app prompts them when the green window opens and shows the saved CO_2 and their streak straight afterwards. Behaviour repeated in the same context for several weeks becomes automatic [11]; at that point the app stops prompting and suggests the next habit.

Home Energy Coach is the team's take on the course topic *Personalized Green Habit Recommender*. It consists of a native iOS app built with SwiftUI (Section III-A), a FastAPI back-end, and an R analytics service that does all data analysis and data mining and is accessed through the `plumber` package. Development follows Scrum over five sprints, with work tracked in Jira and code shared on GitHub. Because the team itself belongs to the target group, the developers can pilot the app on themselves in the final sprint.

A. Objectives

1) Build a platform that recommends personalised home-energy habits to students and renters and prompts them at the right moment, including the greenest hours of the day derived from live German grid data.
2) Use R [1] for all data analysis and data mining tasks: the hourly CO_2 forecast, the recommendation ranking, association rules, clustering and the evaluation of nudges.
3) Deliver at least eight distinct user interfaces beyond standard CRUD and authentication screens; eleven are planned (Section V).
4) Measure the energy and resource consumption of the application itself from the first prototype on, and reduce it in every subsequent sprint.

TABLE I
SCORING OF THE SHORTLISTED DOMAINS (1–5 PER CRITERION)

Candidate *weight*	Real data 30 %	Habit loop 25 %	Screens 15 %	Doable 15 %	Self-test 15 %	Total
Home Energy Coach	5	5	4	4	5	**4.7**
Food Waste Coach	3	5	4	4	5	4.1
Campus Green Challenge	3	4	4	4	5	3.9
Green Grocery Coach	4	4	4	2	4	3.7
Green Commute Coach	4	4	3	4	3	3.7
Digital Footprint Coach	3	3	4	4	4	3.5

5) Follow Scrum [2] with rotating Scrum Master and Product Owner roles, with all progress tracked in Jira.

B. Topic Selection

The course fixes the topic but leaves the application domain open. The team shortlisted six candidate domains and scored them from 1 to 5 against five criteria derived from the course brief: availability of real data (weight 30 %), fit with a cue, routine and reward habit loop (25 %), number of meaningful screens (15 %), feasibility within one semester (15 %) and whether the team can test the app on itself (15 %). Table I shows the result. Home Energy Coach scored highest, mainly because of the free, high-resolution SMARD data and the natural habit loop around appliance timing. The campus challenge idea lives on as the WG and campus challenge feature of Home Energy Coach.

C. Target Users

The primary persona is a student in a shared flat: for example *Lena*, 24, a master's student in a three-person WG in Frankfurt. The flat has gas central heating with thermostatic valves in every room, a shared washing machine and one electricity bill for all three flatmates, so Lena has never seen her own kilowatt-hours. She is motivated but busy and will only follow advice that fits her timetable. The secondary persona is a renter outside a shared flat, for example a young professional in a one-room flat who pays their own bills and is interested in the Green Hours planner and the heating coaching, but not in the group features. Both personas, together with scenarios and user stories, are refined in Sprint 1.

D. Problem Statement

The project has to turn public energy data and a short user profile into habits that people actually keep. That means solving the following problems:

- Most residents of shared flats do not know their own consumption in kWh. There is one bill per flat, usually no smart meter and no submetering, so savings have to be estimated from the profile and from public statistics, with optional meter readings as a refinement.
- Generic advice does not fit. Lowering the thermostat is useless advice in a flat with district heating and no valves, and a laundry tip is pointless without a washing machine. Recommendations have to take the flat type, heating type, available appliances and the user's weekly schedule into account.
- A dashboard on its own does not change routines. The app needs prompts at the right moment, small commitments, immediate feedback and a visible reward, and it has to notice when a habit has formed so that the prompts can stop.
- The same kWh is not always equally green. Whether a load runs at noon or at 19:00 can change its CO_2 footprint considerably, so the app needs current and forecast grid data and has to match green windows to the hours the user is at home.
- Saving energy in a shared flat is a group effort. Heating and hot water are shared, and one flatmate alone cannot save much. The app needs group goals, a WG agreement and challenges, without becoming a way to monitor flatmates.
- There are no real users at the start of the project (the cold-start problem). The recommender has to be built and evaluated on synthetic users derived from public statistics, a survey and a pilot with the team.
- The app has a footprint of its own. An app that asks people to save energy while wasting it on its own servers would contradict its purpose, so its consumption is measured and reduced as part of the project.

E. Scope

In scope. A native iOS app with the eleven screens of Section V; a FastAPI back-end with PostgreSQL; an R analytics service with the pipelines of Section II-E; the Green Hours planner for the German electricity grid, based on SMARD data; a catalogue of 22 home-energy habits covering heating, ventilation, hot water, appliances and standby; WG groups, agreements and challenges; the measurement and reduction of the app's own footprint; and a two-week pilot with the team in Sprint 5.

Out of scope. Android and web clients; automatic control of appliances, smart plugs or thermostats; automatic integration of smart meters or utility accounts (meter readings are entered manually); electricity grids outside Germany; other areas of the course topic such as transport, food and shopping habits; tariff switching and payments; and a public App Store release. Pilot users get the app through TestFlight only. These limits keep the project within the available 1 800 team hours; an Android client and smart-plug integration are noted as future work.

F. Example Scenario

Lena's laundry illustrates the intended five-week path from awareness to habit. In week 0 the onboarding quiz tells her that her WG's laundry causes roughly 4 kg CO_2 per month. In week 1 she commits to a single small goal: one load in green hours on Saturday. On Saturday at 11:30 the app alerts her that a green window opens from 12:00 to 15:00; she starts the machine at noon and sees -0.4 kg CO_2, a streak of one and her WG at rank 2 in the challenge. In weeks 2–4 the goal grows and her flatmates join. From week 5 she runs the machine in green hours without being asked; the prompts fade

TABLE II
SPRINT TIMELINE

#	Dates	Focus
1	26 Oct – 6 Nov	Personas, scenarios and user stories; Jira board and team contract; synthetic-data plan; first SMARD and UBA downloads
2	9 – 20 Nov	Wireframes (Figma), system architecture, UML models, user survey
3	23 Nov – 4 Dec	Back-end and database, R service, SMARD pipeline, first screens
4	7 – 18 Dec	Recommender, Green Hours planner, mockups, green baseline measurement
5	4 – 25 Jan	Pilot with the team, optimisation and re-measurement, final report and delivery

and the app proposes the next habit, lowering the thermostat by 1 °C. All figures are illustrative; the real ones come from SMARD and the user's profile.

II. METHODOLOGY

A. Development Process

The project follows the Scrum framework and is divided into the five sprints prescribed by the course: four two-week sprints and a final three-week sprint, starting on 26 October 2026 and ending with the final delivery on 25 January 2027 (Table II). The practical classes in the first week of each sprint are used for cumulative project work; in the second week the team presents a summary of the completed work. In addition, the team meets the supervisor in the weekly online project meeting. Every sprint contains Sprint Planning, daily stand-ups, a Sprint Review and a Sprint Retrospective; evidence of all four is documented on the Jira board. The course corresponds to 10 ECTS, i.e. about 300 working hours per member or roughly 1 800 team hours for the six members over the 13 project weeks, which the sprint backlogs are sized against.

All tasks are tracked in Jira with the boards Product Backlog, To Do / In Progress / Done, Artifacts, Daily SCRUM and Team Contract; the instructor has access to the project. The team contract defines the expectations for team meetings, team and individual responsibilities, and the strategy for handling impediments and conflicts.

- **Jira project:** mazba09.atlassian.net/jira/software/projects/HEC
- **GitHub repository:** github.com/Asifnewaz/Personalized_Green_habit_Recommender

The Scrum Master (Team Captain) and the Product Owner rotate as shown in Table III. Because the team has six members but the course has only five sprints, the three-week Sprint 5 is split into two role periods, 5a and 5b, of about ten days each. In this way every member holds each role exactly once, and nobody holds both roles in the same period. The split of Sprint 5 is subject to confirmation by the supervisor. The Product Owner maintains and prioritises the product backlog and decides what the sprint delivers; the Scrum Master facilitates the Scrum events, keeps the board up to date and removes impediments.

TABLE III
ROTATION OF THE SCRUM ROLES

Sprint	Dates	Scrum Master	Product Owner
1	26 Oct – 6 Nov	Mazba Uddin	Asif Newaz
2	9 – 20 Nov	Madhusudan Ambulkar	Muhammad Saad Zafar
3	23 Nov – 4 Dec	Md. Rabiul Hasan Tanvir	Nazmul Alam
4	7 – 18 Dec	Nazmul Alam	Madhusudan Ambulkar
5a	4 – 14 Jan	Muhammad Saad Zafar	Mazba Uddin
5b	15 – 25 Jan	Asif Newaz	Md. Rabiul Hasan Tanvir

TABLE IV
AREAS OF RESPONSIBILITY

Area	Main responsibilities	Members
R analytics	SMARD pipeline and Green Hours forecast, recommender, association rules, clustering, evaluation; `testthat` tests	Ambulkar, Tanvir
Back-end and database	FastAPI services, API contract, authentication, PostgreSQL schema, scheduler, R bridge; `pytest` tests	Uddin, Zafar
iOS app	SwiftUI screens, local and push notifications, offline cache, accessibility; XCTest and XCUITest	Newaz
UX and design	Personas, scenarios, survey, Figma wireframes and mockups, usability tests in the pilot	Newaz, Tanvir
Architecture and modelling	Architecture document and its justification; use case, class, activity and sequence diagrams	Zafar, Newaz, Ambulkar
Data and synthetic users	Open-data downloads (UBA, Stromspiegel, REFIT), habit catalogue figures, synthetic users, survey analysis	Ambulkar, Alam
Green software	Baseline and per-sprint measurements, optimisation backlog, data for the "How green is this app?" screen	Uddin, Alam
DevOps and quality	Docker Compose, GitHub Actions, test strategy, code-review rules, Definition of Done	Zafar, Newaz
Documentation	Proposal, user documentation, final report, Artifacts board in Jira	Alam, Tanvir

B. Team Responsibilities

Besides the rotating Scrum roles, the work is divided into technical and organisational areas (Table IV). An area owner is accountable for the quality and timely delivery of the area and is the first point of contact for it, but does not do all of its work alone. A member can own or contribute to several areas, and an area can have several owners; the R analytics, the largest and most specialised area, has at least two, so that the knowledge does not depend on a single person. Every member writes code and reviews other members' pull requests. Members are listed by surname; the assignment is also recorded in the team contract in Jira.

C. Data Collection and Integration

The application combines three kinds of data: open data about the German electricity system, statistics about typical

TABLE V
DATA SOURCES

Need	Source	Use in the app
Grid CO_2 by hour	SMARD (Bundesnetzagentur): generation by source and consumption, hourly, data since 2015, free [3]	Green Hours forecast in R
kWh $\rightarrow$ CO_2	UBA emission factor of the German electricity mix: 344 g CO_2/kWh (2025, estimate) [4]	Default conversion for savings
"Similar flats" baseline	Stromspiegel 2025 (co2online, about 57 000 electricity bills) [5]	Onboarding comparison, savings estimates
Split by end use	UBA household energy data (space heating $> 2/3$) [6]	Which habits matter most
When appliances run	REFIT: 20 UK homes, 2 years, appliance-level loads [7]	Realistic synthetic users
Habit adoption	Synthetic users, survey, Sprint 5 pilot	Training and evaluation of the recommender

households, and the data generated by its own users. Table V lists the sources and their use.

SMARD provides the actual generation per energy source (lignite, hard coal, gas, wind, solar, hydro, biomass, ...), the day-ahead generation forecast and the actual consumption for the German market area in hourly and quarter-hourly resolution. The data goes back to 2015, is free of charge and is available through JSON endpoints, which can be queried from R either directly with `httr2` or through the CRAN package `bunddev`, which wraps the SMARD endpoint with caching and rate limiting [8]. The hourly CO_2 intensity of the grid is computed from the generation mix and the per-source emission factors; the yearly average published by the Umweltbundesamt (344 g CO_2/kWh for 2025 as an estimate, 353 g for 2024 [4]) serves as the default conversion wherever no hourly value is needed, e.g. for yearly savings. The corresponding UBA factors for natural gas and district heating are used for heating habits.

Household statistics give the app a baseline before a user has produced any data. The Stromspiegel 2025 of co2online, based on about 57 000 electricity bills, provides typical consumption by household size and building type [5]; it powers the "similar flats" comparison during onboarding and the estimate of what a habit saves. The UBA statistics on the energy consumption of private households provide the split by end use (space heating, hot water, cooking, appliances), which shows which habits matter most [6]. The REFIT dataset, appliance-level load measurements of 20 UK households over two years [7], provides realistic patterns of when washing machines, dishwashers and other appliances actually run; it is used to generate plausible synthetic users for the cold-start phase.

User-generated data consists of the onboarding quiz, commitments, check-ins, dismissed tips, optional meter readings and notification responses. A short survey among fellow students in Sprint 2 and the pilot with the team in Sprint 5 provide the first real habit-adoption data. An exploratory data analysis in R during Sprints 1 and 2 examines consumption distributions, the daily and seasonal shape of the CO_2 intensity curve and typical appliance run times before any modelling decision is made.

D. Data Preprocessing Pipeline

All preprocessing is implemented in R (`dplyr`, `tidyr`, `lubridate`, `httr2`, `jsonlite`) inside the analytics service, so that the same code serves the exploratory phase and the production pipeline:

- Every night a job downloads the actual generation per source for the past day and the generation forecast for the next day. It resamples quarter-hourly values to hours, aligns timestamps to Europe/Berlin (including daylight-saving changes), carries values forward over gaps of up to two hours, flags longer gaps, and computes the hourly CO_2 intensity as the emission-factor-weighted mean of the generation mix. The result is stored in PostgreSQL, and the last successful download is kept as a fallback snapshot.
- The quiz answers are normalised into a feature vector: flat size, number of flatmates, heating and hot-water type, available appliances, and the weekly home schedule as a 24×7 availability matrix. Consumption in shared flats is expressed per person so that WGs of different sizes can be compared.
- Each of the 22 habits in the catalogue is stored with its type (timing, reduction or one-time), the estimated saving per execution in kWh, CO_2 and euros, an ease score, its prerequisites (e.g. a washing machine, or a thermostat of one's own) and the app tool that supports it (Table VI).
- For the cold start, synthetic profiles are drawn from the Stromspiegel consumption bands and household sizes, the UBA end-use split and the REFIT appliance run times. Their commitment and check-in histories are simulated over several weeks and later replaced by real check-ins from the survey and the pilot.
- Duplicate check-ins are removed and implausible meter readings are flagged. The analytics service only works with pseudonymous user IDs, and users can export or delete their data from the privacy screen.

E. Data Mining and Recommendation Models (R)

All modelling runs in the separate R service, which the FastAPI back-end calls through `plumber`. It consists of six pipelines.

1) **Green Hours forecast.** The service fetches the generation mix and the day-ahead forecast from SMARD with `httr2` or `bunddev`, resamples quarter-hourly entries to hours, aligns them to Europe/Berlin with `lubridate` and fills gaps of up to two hours by carrying values forward (`dplyr`). The hourly carbon intensity in g CO_2/kWh is the generation mix weighted by the official UBA emission factors. The forecast is then matched against the user's home schedule to

find the cleanest hours for flexible appliances such as washing machines and dishwashers.

2) **Recommendation ranking.** At onboarding, candidate habits are filtered by the user's prerequisites, such as heating type and available appliances, and scored with a multi-attribute utility heuristic based on estimated CO_2 savings, ease and fit with the schedule (`dplyr`). Once check-ins and survey records accumulate, an item-based collaborative filtering (IBCF) model in `recommenderlab` computes habit similarities from the matrix of which users completed which habits, and recommends habits that worked for similar profiles.
3) **Habit stacking.** Check-in logs are grouped into routine-based transactions, and the Apriori algorithm in `arules` finds habits that recur together, using thresholds for support, confidence and lift. When a user keeps up one habit (e.g. *Stoßlüften*), rules with a lift above 1 trigger a nudge for a linked habit (e.g. lowering the thermostat).
4) **Persona clustering.** Each user is described by a normalised vector of daytime home availability, completion rates per habit type and notification dismissal rates. Users are grouped with k-means (base `kmeans` in `stats`), with k-medoids (`pam` from `cluster`) as a variant that copes better with outliers. Silhouette analysis with `factoextra` sets the number of clusters, which might separate daytime from weekend savers, for example. The cluster centres are used to tailor the default nudges for new users.
5) **Evaluation and nudge testing.** Recommender accuracy is measured with 5-fold cross-validation on synthetic and survey data, using Precision@k, Recall@k and MAE from `recommenderlab`. To see whether nudges produce lasting habits, reminders are faded after an initial period of consistent check-ins, and a paired t-test or Wilcoxon signed-rank test (`stats`) compares check-in adherence before and after the fade.
6) **Serving and efficiency.** The `plumber` endpoints answer only fast lookups and reach the database through `DBI` and `RPostgres`. Data-heavy jobs such as SMARD downloads, clustering and matrix recomputation run as scheduled nightly batches to keep server CPU use low. Every R function is profiled in each sprint with `bench` and `profvis` to bring runtime and memory use down.

F. Behaviour Change and Personalisation Design

Personalisation happens at three levels. On day one, the *profile* from the onboarding quiz filters the habit catalogue: only habits whose prerequisites the flat fulfils are shown. After one to two weeks, the user's *behaviour* (check-ins, dismissals, response times to prompts) assigns them to a persona cluster, and tips that worked for similar users are preferred (Section II-E). On an ongoing basis, *feedback* adjusts the feed: dismissed tips fade out and mastered habits unlock the next step. Personalisation also applies to time: a user who is at home from 18:00 to 22:00 is told that 19:00 is their greenest slot, not that noon would have been better.

TABLE VI
HABIT CATALOGUE BY TYPE

Type	#	Examples	App tool
Timing habits	4	Laundry, dishwasher, charging and batch cooking in green hours	Real-time Green Hours prompt
Reduction habits	12	Heating $-1\,^{\circ}$C, Stoßlüften, shorter showers, 30 °C wash, standby off	Habit loop: commit, check in, streak
One-time actions	6	LEDs, smart plug, bleeding radiators, green electricity tariff, WG agreement	Checklist with lifetime saving

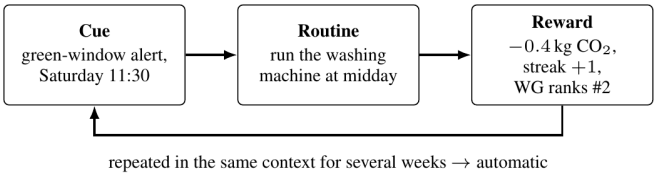


Fig. 1. The habit loop on which every screen of the app is based.

The 22 habits fall into three types that need three different tools (Table VI). Timing habits are driven by the real-time Green Hours prompt; reduction habits run through the habit loop of commitment, check-in and streak; one-time actions are a checklist whose lifetime saving is credited once. Every screen maps to one mechanism of the habit loop shown in Fig. 1: the dashboard is feedback, the commitment is an implementation intention, the notification is the prompt, and the streak and the WG ranking are the reward.

Nudging raises ethical questions, and the course brief explicitly asks for ethical AI and transparency. Home Energy Coach therefore uses only opt-in nudges: users choose their own commitments, every recommendation comes with a "why this" explanation, notifications can be muted per habit, and prompts stop automatically once a habit is established. The WG challenge shows the flat's shared score rather than who did what, so that flatmates do not put pressure on each other. The app uses no dark patterns such as artificial scarcity, guilt messages or hidden opt-outs.

G. Testing and Quality Assurance

Testing starts in Sprint 3, as soon as the first components exist, and works at three levels. Unit tests cover the R functions (`testthat`), the back-end services (`pytest`) and the iOS view models (XCTest). Integration tests cover the bridge between FastAPI and `plumber` and the SMARD ingestion; they use recorded API responses as fixtures so that they do not depend on the live service. End-to-end tests run the main user flows, from onboarding through recommendation and commitment to check-in, with XCUITest on the iOS app. GitHub Actions runs the test suites on every pull request, and

Docker gives every member the same test environment. In Sprint 5 the team members use the app for two weeks as a pilot. This gives the first real check-in data, usability feedback and the before-and-after measurements for the evaluation in Section II-E.

H. Green Software Engineering: the App's Own Footprint

The course treats the app's own greenness as a central part of the assessment. The team works in three steps that cover the points named in the course brief (internal baselines, comparison with common practice, algorithmic efficiency and documented design choices) and uses the Software Carbon Intensity specification of the Green Software Foundation to choose its metrics [12].

1) **Baseline (Sprint 4).** The first prototype is measured on the phone with the Xcode Energy Gauge and Instruments, on the server (CPU time, memory and network bytes per request, taken from Docker statistics and API metrics) and in R (runtime and memory of each analytics job, measured with `bench` and `profvis`). The baseline is documented and sets the targets for the following sprints.
2) **Optimisation.** Analytics run as nightly batch jobs instead of per request. SMARD data, the daily green-window forecast and the recommendations are computed once and cached, and payloads are kept small and paginated. Green-window alerts are scheduled locally on the phone, and only for windows that match the user's schedule. Alternative algorithms for the models in Section II-E are compared on runtime and memory as well as on accuracy.
3) **Re-measurement.** The measurements are repeated in every sprint, and the trend is recorded and shown on the in-app screen "How green is this app?". The final report documents each design decision together with its measured effect.

III. System Architecture

The platform uses a **layered client-server architecture with a separate analytics service** (Fig. 2). It has four layers: the native iOS app, a back-end built with FastAPI, an analytics layer in R and a data layer. Components communicate over REST endpoints with JSON payloads, and the app never talks to R directly. The course requires R for all analysis tasks, and putting R in its own service keeps the general-purpose back-end simple. It also lets the analytics run as scheduled batch jobs, which is the most energy-efficient option, and allows different team members to develop and test the back-end and the analytics independently. The architecture document in Sprint 2 records and justifies this choice.

A. Frontend Layer

The front-end is a native iOS app written in Swift with SwiftUI [15]. The team compared it with a web application (React, installable as a progressive web app) on four criteria: existing skills, notifications, energy measurement tooling and device reach. iOS came out ahead on the first three. The Green Hours planner only works if the alert arrives at the right moment, and iOS supports scheduled local notifications natively. Once the daily forecast has been fetched, the app schedules the user's green-window alerts on the phone, so the server does not have to send a push message for every window. For measurement, the Xcode Energy Gauge and Instruments break the app's energy impact down into CPU, network, GPU and other overhead, which feeds directly into the green software evaluation. And part of the team develops iOS apps professionally, which lowers the risk of the largest user-interface workload in the project.

The drawback is reach: flatmates with Android phones cannot use the app in this project. Because the app talks only to the back-end's REST API, whose contract is defined first, a web or Android client can be added later without changes on the server. The app contains the eleven screens of Section V, caches the current forecast and recommendations so that it also works offline, and reaches the pilot users through TestFlight [17].

B. Backend Layer

The back-end is written in Python with FastAPI [14]. An API gateway handles routing, JWT authentication and request validation. Domain services manage user profiles and flats, the habit catalogue, commitments and check-ins, WG groups with their agreements and challenges, and notifications. A scheduler triggers the nightly SMARD fetch, forecast and recommendation refresh in the R service. Since green-window alerts are scheduled on the phone, the server sends push notifications only for events such as WG challenge updates. An R bridge calls the `plumber` endpoints over HTTP and stores the results in PostgreSQL, so that the API serves cached results instead of recomputing them for every request.

C. Analytics Layer

The R analytics service exposes its functions through `plumber` [13]: endpoints for the green-hours forecast, recommendations, personas, association rules and the evaluation reports. It runs in batch mode once per night and on demand only for small requests such as a user's personal green window after a schedule change. The service is stateless, reads from PostgreSQL through a read-only database role (`DBI`/`RPostgres`) and writes its results to dedicated tables. The packages used for the models are chosen and justified in Section II-E.

D. Data Layer

PostgreSQL holds users, flats (WGs) and memberships, the habit catalogue, commitments, check-ins, notifications, the hourly grid-intensity cache, recommendations and the green measurements of the app itself. The UBA, Stromspiegel and REFIT extracts are versioned in the repository as CSV files and loaded by the R service. The system stores as little personal data as possible (e-mail address and display name), uses pseudonymous IDs in the analytics service, and lets users export or delete all of their personal data in the app.

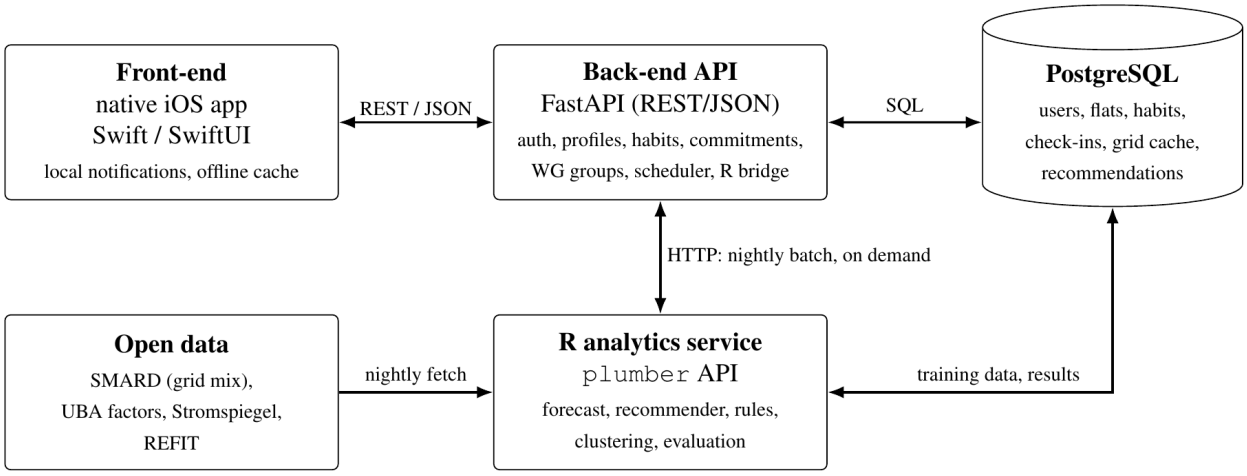


Fig. 2. High-level system architecture: a layered client-server system in which R runs in its own analytics service. All server-side components run as Docker containers.

E. Containerisation and Deployment

The server side is containerised with Docker [16] and orchestrated with Docker Compose. Each major component runs as an isolated service:

- `backend`: Python container running FastAPI with Uvicorn.
- `r_service`: R container exposing the `plumber` endpoints.
- `scheduler`: triggers the nightly jobs and the notification run.
- `postgres`: persistent relational storage.

A single `docker-compose.yml` at the repository root defines all services, networks and volumes, so that every member and the instructor can start the full stack with one command. GitHub Actions builds the images and runs the tests on every pull request. The iOS app is not containerised: it is built and tested with Xcode on a macOS runner and distributed through TestFlight.

IV. Technology Stack

Table VII summarises the technologies per layer. The stack follows the architecture: one general-purpose language for the API, R for everything analytical, and a relational database as the single source of truth. All server-side tools are free, run on macOS, Windows and Linux and are available as Docker images, so that back-end and R work does not depend on a particular machine. iOS development requires a Mac with Xcode, so the iOS area is assigned to members who have one.

TABLE VII
Technology Stack

Layer	Technology	Purpose
Front-end	Swift, SwiftUI (native iOS)	User interface, local notifications
Back-end	Python (FastAPI)	REST API, scheduling, R bridge
Analysis	R (`plumber`)	Forecast, recommender, clustering, evaluation
Database	PostgreSQL	Persistent storage, caches
Deployment	Docker Compose, GitHub Actions	Containers, CI
Design	Figma	Wireframes and mockups
Modelling	draw.io / PlantUML	UML diagrams
Management	Jira, GitHub	Backlog, Scrum evidence, code

V. User Interfaces

Eleven interfaces beyond basic CRUD and authentication are planned. That is three more than the eight required, so the scope can shrink without falling below the requirement:

1) **Onboarding energy quiz**: flat size, flatmates, heating type, appliances and weekly schedule, with an immediate "similar flats" comparison.
2) **Energy dashboard**: estimated monthly kWh, CO_2 and euro per end use, and the progress of the current habits.
3) **Recommendation feed with "why this"**: ranked habits, each with the reason it was chosen for this user.
4) **Habit commitment and check-in**: choose a goal, define when and where it happens, log completions, see the streak.
5) **Green Hours planner**: today's and tomorrow's grid CO_2 curve with the user's personal green windows and reminders.
6) **Habit library**: all 22 habits with saving estimates, filterable by type and effort.
7) **One-time checklist**: one-off actions with their lifetime saving.
8) **WG agreement and challenge**: shared goals and house rules of the flat and the ranking between flats.
9) **Trends and analytics**: weekly and monthly development, before/after comparison of adopted habits.

TABLE VIII
SUCCESS CRITERIA

Area	Criterion and target
Functionality	At least 8 of the 11 planned screens work end-to-end against the live back-end; target: all 11
R analytics	Forecast, recommendation ranking, association rules, clustering and evaluation all run in R in the nightly batch, each covered by `testthat` tests
Green Hours	In a back-test over at least four weeks of SMARD data, the forecast greenest 3-hour window overlaps the actual greenest window on at least 70 % of days
Recommender	In 5-fold cross-validation on synthetic and survey data, the personalised ranking reaches a higher Precision@5 than a popularity baseline
Habit formation	In the pilot, at least two thirds of the participants keep a committed habit with at least 70 % check-in adherence over two weeks; adherence after the prompts fade is reported with a paired test
Usability	Mean System Usability Scale (SUS) score of at least 68, the commonly used average, in the pilot [18]
Green software	Median server CPU time and data transferred per user session at least 30 % lower in the final version than at the Sprint 4 baseline; no model computed per request; "Low" energy impact of the iOS app in a typical session
Process	Every member has been Scrum Master once and Product Owner once; planning, stand-up, review and retrospective evidence in Jira for every sprint; all merged code meets the Definition of Done

10) **Privacy and data centre**: what is stored, export and deletion, notification settings.
11) **"How green is this app?"**: the measured footprint of the application itself, sprint by sprint.

Wireframes are developed in Figma in Sprint 2 and refined into mockups in Sprint 4.

VI. Success Criteria

The project is successful if the criteria in Table VIII are met at the final delivery. The quantitative targets are initial values: they are reviewed once the first data and the green baseline are available in Sprint 4, and any change is documented in Jira. Because the pilot involves only the six team members, its results are indicative rather than statistically conclusive.

VII. Risks and Mitigation

Table IX lists the main risks identified during topic selection and how the team intends to handle them. The risk register is kept in Jira and reviewed in every Sprint Retrospective.

TABLE IX
RISKS AND MITIGATION

Risk	Mitigation
No real users to train models on	Synthetic users from Stromspiegel, UBA and REFIT; survey in Sprint 2; pilot with the team in Sprint 5
Users cannot measure their own kWh	Estimate from the profile and public statistics; optional meter readings
SMARD API changes or downtime	Nightly caching; fallback to the stored dataset; ingestion isolated in one R module
R skills uneven across the team	Two R owners; shared R scripts; pair-programming sessions
Nudges feel manipulative or annoying	User-chosen commitments, mute option, "why this" on every tip, prompts fade once a habit is formed
iOS-only app excludes flatmates with Android phones	Client-independent REST API defined first, so a web or Android client can be added later; Android support listed as future work
Scope too large for one semester	Eight screens required, eleven planned; the additional screens are the first candidates to be descoped

VIII. Deliverables

1) Personas, scenarios and user stories
2) Wireframes and mockups (Figma)
3) System architecture document with justification of the architectural style
4) UML models: use case, class, activity and sequence diagrams
5) Working native iOS app with back-end and R analytics service; the server side containerised with Docker
6) Green software evaluation: baseline, optimisations and measured trend of the app's own footprint
7) Jira project with Scrum evidence, artifacts and team contract, accessible to the instructor
8) User documentation and final project report

IX. Conclusion

Home Energy Coach is aimed at people whom most energy advice ignores: students and renters in shared flats, who cannot renovate their building but can change when and how they use energy. It combines public data on the German electricity grid with a short user profile to recommend habits that fit each flat and each schedule, and it uses small commitments, prompts at the right moment and immediate feedback to turn those habits into routines. All analysis runs in a separate R service, as the course requires, and as nightly batch jobs, which also keeps the app's own footprint low. The defined scope, the measurable success criteria, the role rotation for all six members and the pilot with the team are meant to let the final delivery report measured results for both the recommender and the greenness of the software.

References

[1] R Core Team, *R: A Language and Environment for Statistical Computing*, R Foundation for Statistical Computing, Vienna, Austria, 2025. [Online]. Available: https://www.R-project.org/

[2] K. Schwaber and J. Sutherland, "The Scrum Guide," 2020. [Online]. Available: https://scrumguides.org/

[3] Bundesnetzagentur, "SMARD – Strommarktdaten: Electricity market data for Germany," data download and API. [Online]. Available: https://www.smard.de/

[4] Umweltbundesamt, "Entwicklung der spezifischen Treibhausgas-Emissionen des deutschen Strommix in den Jahren 1990–2025," Dessau-Roßlau, Mar. 2026. [Online]. Available: https://www.umweltbundesamt.de/themen/co2-emissionen-pro-kilowattstunde-strom-2025-nur

[5] co2online gGmbH, "Stromspiegel 2025: Deutsche Haushalte verschwenden jährlich 12 Milliarden Euro für Strom," press release, Berlin, May 2025. [Online]. Available: https://www.stromspiegel.de/

[6] Umweltbundesamt, "Energieverbrauch privater Haushalte," data and Excel downloads, 2026. [Online]. Available: https://www.umweltbundesamt.de/daten/private-haushalte-konsum/wohnen/energieverbrauch-privater-haushalte

[7] D. Murray, L. Stankovic, and V. Stankovic, “An electrical load measurements dataset of United Kingdom households from a two-year longitudinal study,” *Scientific Data*, vol. 4, art. 160122, 2017, doi: 10.1038/sdata.2016.122.

[8] *bunddev: R Client for the bund.dev Public APIs*, R package, CRAN, 2026. [Online]. Available: https://cran.r-project.org/package=bunddev

[9] B. J. Fogg, “A behavior model for persuasive design,” in *Proc. 4th Int. Conf. Persuasive Technology (Persuasive ’09)*, Claremont, CA, USA, 2009, art. 40.

[10] P. M. Gollwitzer, “Implementation intentions: Strong effects of simple plans,” *American Psychologist*, vol. 54, no. 7, pp. 493–503, 1999.

[11] P. Lally, C. H. M. van Jaarsveld, H. W. W. Potts, and J. Wardle, “How are habits formed: Modelling habit formation in the real world,” *European Journal of Social Psychology*, vol. 40, no. 6, pp. 998–1009, 2010.

[12] Green Software Foundation, “Software Carbon Intensity (SCI) Specification,” ISO/IEC 21031:2024. [Online]. Available: https://sci.greensoftware.foundation/

[13] B. Schloerke and J. Allen, *plumber: An API Generator for R*, R package. [Online]. Available: https://www.rplumber.io/

[14] S. Ramírez, “FastAPI: Modern, fast web framework for building APIs with Python.” [Online]. Available: https://fastapi.tiangolo.com/

[15] Apple Inc., “SwiftUI.” [Online]. Available: https://developer.apple.com/xcode/swiftui/

[16] Docker, Inc., “Docker: Accelerated container application development.” [Online]. Available: https://www.docker.com/

[17] Apple Inc., “TestFlight.” [Online]. Available: https://developer.apple.com/testflight/

[18] J. Brooke, “SUS: A ‘quick and dirty’ usability scale,” in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds. London, U.K.: Taylor & Francis, 1996, pp. 189–194.